

Experiment-1

AIM- To configure a GPIO pin as digital output and implement LED blinking to illustrate basic edge-node operation.

APPARATUS- STM32 Discovery Kit, STM32 CUBE IDE, STM32 CubeMX.

PROCEDURE

1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.
2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L475VGT6.
3. In system Core select GPIO and right click on pin PB14 to make it as GPIO output pin as shown below as LED (LED2) relates to it in the board.

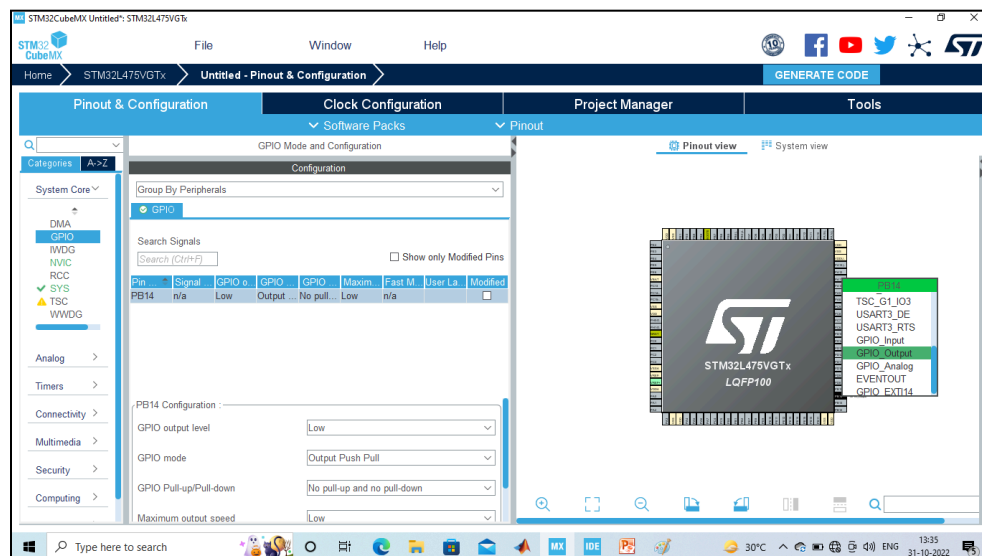


Fig-1.1 – STM32 Board Pinout and Configuration

4. **Clock Selection:** Select the internal clock by clicking RCC and selecting BYPASS clock source

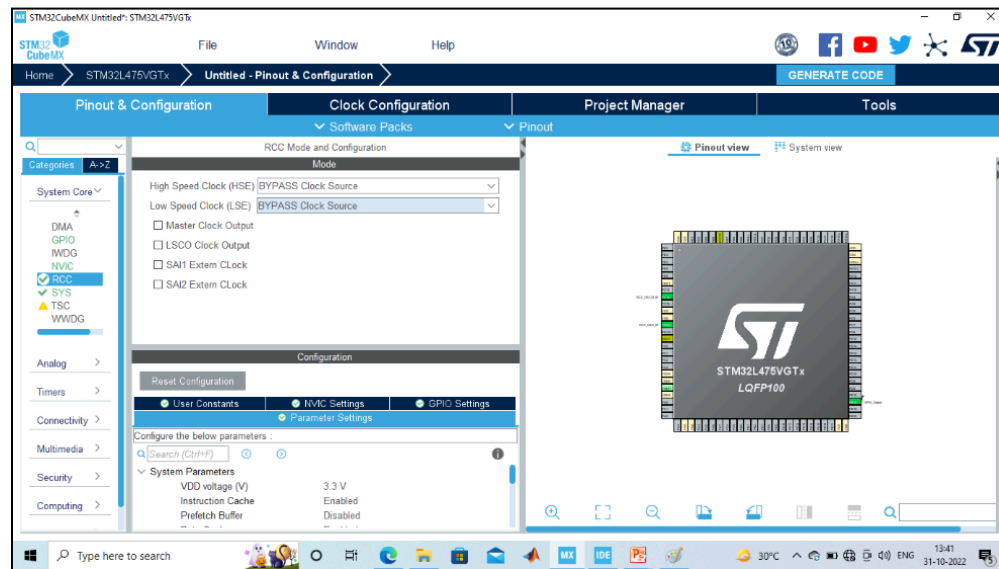


Fig-1.2 – Clock Selection

5. **Clock Configuration:** Configure clock as shown below to get maximum internal clock frequency. Set LSE at 32KHz and HSE at 16MHz to get maximum frequency 80MHz.

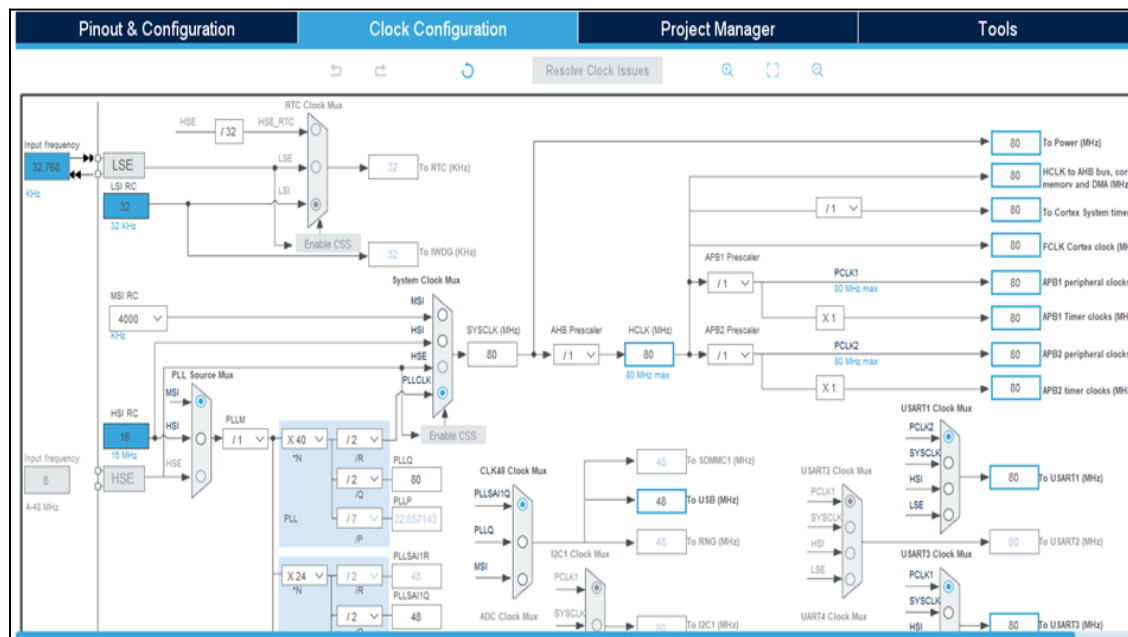


Fig-1.3 – Clock Configuration

6. Write Project name and select IDE.

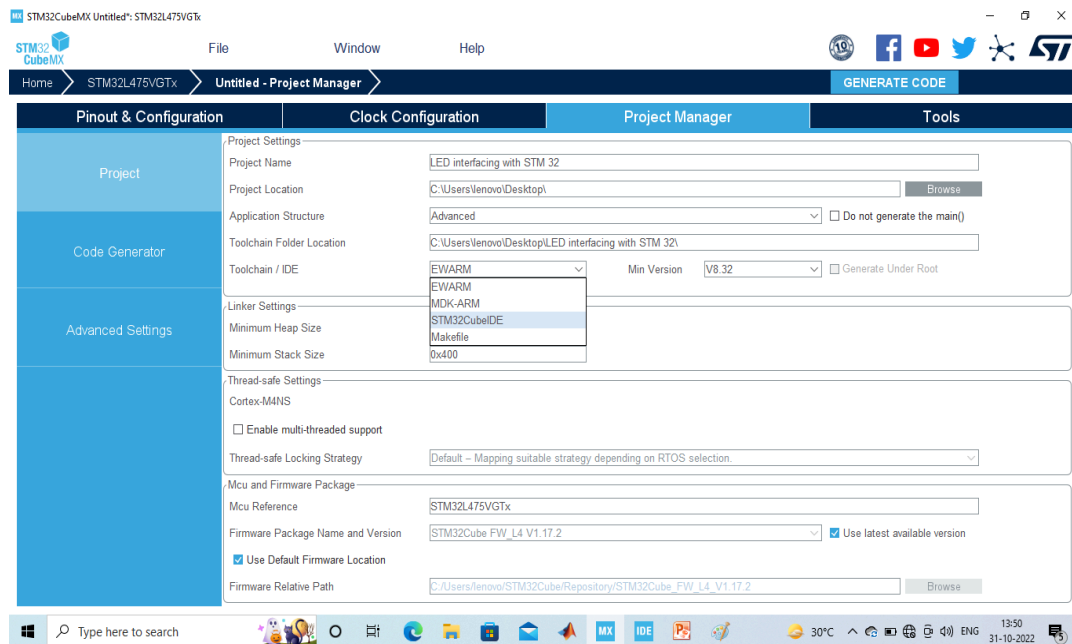


Fig-1.4 – Project Name and IDE

7. Click on generate code.

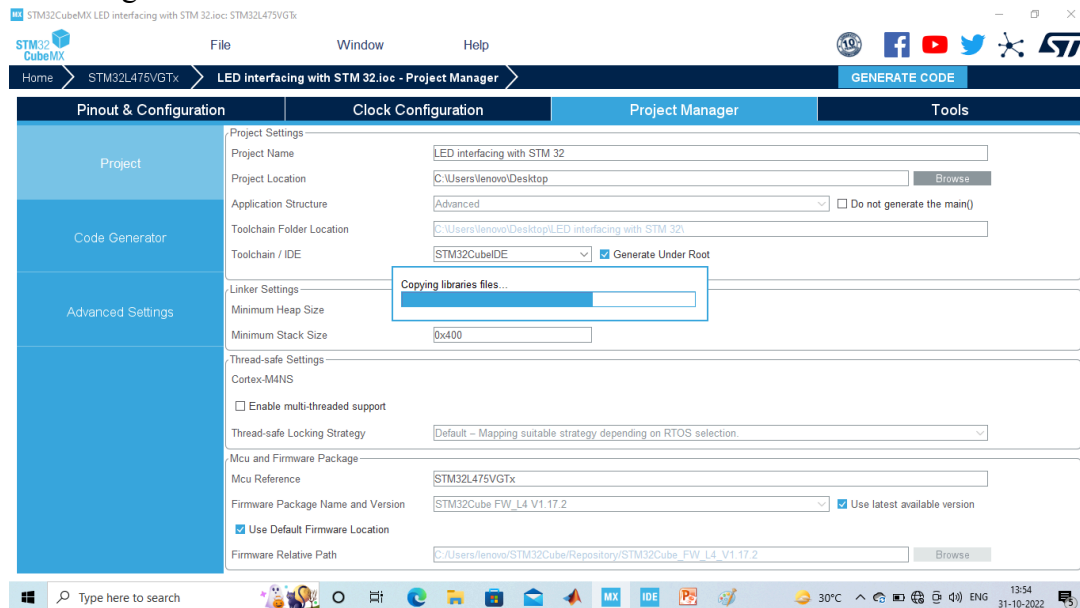


Fig-1.5 – Generate Code

8. Click on open project.

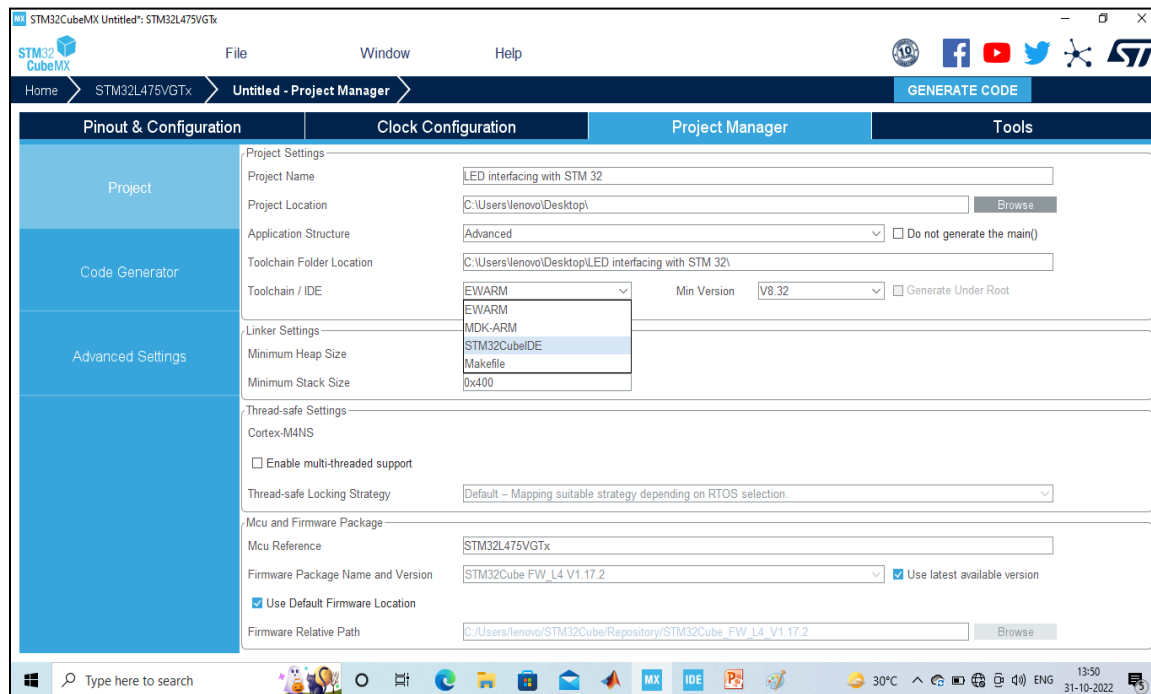


Fig:1.6 – Project Set up

9. Click 'OK' on the generated popup showing "successfully imported the project on the work space" and the project will be successfully added to the cube IDE.
10. Open the main source code on Cube IDE by clicking on **main.c**
11. To generate bin file and hex file, right click on project and click properties.
12. Expand C/C++ Build, click on setting, click on MCU post build outputs and select convert to binary file as well as convert to hex file and click "Apply and Close".
13. After configuration write only below two instructions in the user while loop and compile the program and ensure there will be zero error after compiling.

```
HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_14);
HAL_Delay(1000);
```



```
workspace_1.10.1 - LED interfacing with STM32/Core/Src/main.c - STM32CubeIDE
File Edit Source Refactor Navigate Search Project Run Window Help

Project Explorer X
> LED interfacing with STM32

main.c X
07 // Initialize all configured peripherals //
86 MX_GPIO_Init();
87 /* USER CODE BEGIN 2 */
88
89 /* USER CODE END 2 */
90
91 /* Infinite loop */
92 /* USER CODE BEGIN WHILE */
93 while (1)
94 {
95     /* USER CODE END WHILE */
96     HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_14);
97     HAL_Delay(1000);
98     /* USER CODE BEGIN 3 */
99 }
100 /* USER CODE END 3 */
101 }
102
103 /**
104  * @brief System Clock Configuration
105  * @retval None
106  */
107 void SystemClock_Config(void)
108 {
109     RCC_OscInitTypeDef RCC_OscInitStruct = {0};
110     RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
111
112     /** Configure the main internal regulator output voltage
```

Fig: 1.7 - Program for executing the experiment

14. To blink the connected LED on the board, connect the board, click on build, click 'ok' and click 'switch' on popups.
15. Now you are in debugging mode, click 'Resume' to execute the program.

OBSERVATION: Change the delay value (e.g., 200 ms, 1000 ms), re-build, re-run, and note the change in blink speed and fill the observation table.

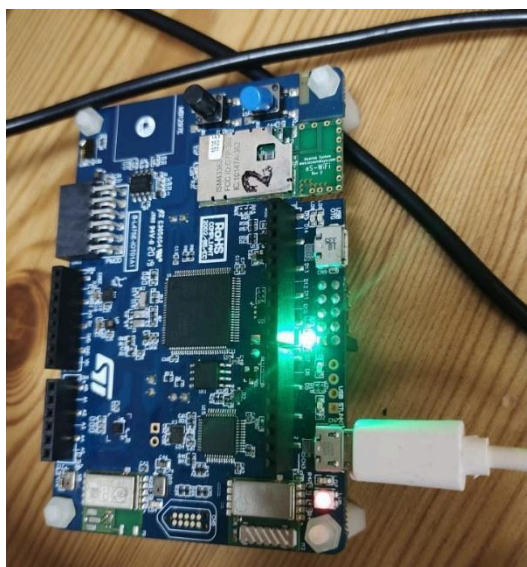


Fig: 1.8 – Output from experiment (Blinking LED)



S. No.	GPIO pin used	Mode / Type / Pull	Delay value (ms)	Approx. blink period (s)	Comment (slow/fast/visible)
1	PA5	Output, PP, No PU/PD	100	0.2	Very Fast, Blink is not clearly visible
1	PA5	Output, PP, No PU/PD	200	0.4	Fast, Blink is visible
2	PA5	Output, PP, No PU/PD	500	1.0	Moderate, Blink is easily visible
3	PA5	Output, PP, No PU/PD	1000	2	Slow, Blink is easily visible

Write down reflection summary (min 200 words) on your understanding of GPIO configuration parameters of STM32 MCU's?

This experiment shows how the GPIO pins works in STM32 microcontroller. GPIO pins are used to connect the microcontroller with external devices like LEDs, sensors, and buttons. In this task, one GPIO pin was set as a digital output to make an LED blink. This shows how a microcontroller can send a HIGH or LOW signal to control a device.

STM32CubeMX helps in selecting the pin and choosing its mode. The pin was set as an output pin. The experiment also involved setting the clock, which is important because the timing of delays and program execution depends on the clock frequency.

After selecting the pin, its mode and the clock frequency the code is generated for the led blink program. The two main command in this experiment are `HAL_GPIO_TogglePIN(GPIOB, GPIO_PIN_14)` and `HAL_Delay(500)`.

The `HAL_GPIO_TogglePin()` command changes the state of the LED, if the LED is ON it will turn it OFF and If the LED is OFF it will turn it ON. This repeated toggling creates the Blinking effect of the LED.

The second Command used is `HAL_Delay()`. This function pauses the program for a specific period of time, which creates a delay between each toggle.

Overall, the experiment gives a simple understanding of the basic GPIO settings such as pin mode, output type, and clock frequency. It also shows that the clock settings affect how the GPIO works.